The values of rational probabilistic finite automata are the left-c.e. reals

Seth Douglas Nidhal Mghirbi

Version 1.0.2, November 2026

Abstract

Which real numbers are values of probabilistic finite automata with rational transition probabilities and a rational initial distribution? We show that they are exactly the left-c.e. reals in $[0, 1]$, the limits of computable nondecreasing sequences of rational numbers. Here the value of an automaton is the supremum, over all finite input words, of the probability that the word is accepted. Every value is left-c.e. because the words can be enumerated; we prove that every left-c.e. real in $[0, 1]$ occurs, and that two input letters and one fixed number of states suffice for all of them.

In particular a single automaton can have a noncomputable value. It was known that no algorithm computes, or even approximates, the value from the automaton; that concerns the map from automata to values, not single values. We give an automaton, specified explicitly relative to a fixed enumeration of Turing machines, whose value is $1/4$ plus the sum of $4^{-(n+2)}$ over the programs n that halt on input n. The automaton checks claimed runs of a counter machine by a likelihood test in the manner of Condon and Lipton, and pays out a reward read from each run's output. The construction uses a rational interval around the target; from the target's approximations alone no algorithm can produce the automaton, since an automaton of value 0 can be recognized. The halting probability of every prefix-free machine, Chaitin's Ω among them, is the value of a probabilistic finite automaton with two input letters. With a construction of Elkouss and Pérez-García, every left-c.e. real in $[0, 1]$ is also the capacity of a finite-state channel with rational transition probabilities; in particular one such channel has a noncomputable capacity.

AI systems assisted with this work; see "How this work was done."

Contents

1 Introduction **2**
1.1 How the automaton works . . . 3
1.2 Prior work . . . 4

2 Preliminaries **5**

3 Counter programs that output words **6**

4 Checking a run with two random tests **8**
4.1 The equality test . . . 8
4.2 Certificates . . . 9
4.3 The parser . . . 9
4.4 The checking automaton . . . 10

5 The payout and the proof of Theorem A **11**

6 An explicit noncomputable value and halting probabilities **13**

7 Two letters and a fixed number of states 14

8 Uniformity 16

9 Open problems 16

How this work was done 18

Verification status 19

Credit 20

1 Introduction

A probabilistic finite automaton, introduced by Rabin [21], has finitely many states. On each input letter it moves from its current state to a random next state, with probabilities given by a stochastic matrix for that letter, and it accepts a word if it ends in an accepting state. We consider automata whose transition probabilities and initial distribution are rational. The *value* $V(A)$ of such an automaton A is the supremum, over all finite words w, of the probability $P_A(w)$ that A accepts w. Which real numbers are values?

A real number is *left-c.e.* if it is the limit of a computable nondecreasing sequence of rational numbers [7]. Every value is left-c.e.: each $P_A(w)$ is a rational number computable from A and w, and the maximum of P_A over the first n words of an enumeration increases to $V(A)$ (Proposition 2.3). This observation is folklore; it amounts to the semi-decidability of the strict emptiness problem, whether $V(A) > \lambda$ for a given rational λ. Our main result is the converse.

Theorem A. The values of rational probabilistic finite automata are exactly the left-c.e. reals in $[0, 1]$.

Theorem B. There is a number N such that every left-c.e. real in $[0, 1]$ is the value of a rational probabilistic finite automaton with two input letters and at most N states.

The number N is fixed by one universal counter program (Section 7); we have not computed it, and it is large. Two states do not suffice: an automaton with two states has a rational value (Proposition 9.1).

It has long been known that the value cannot be computed from the automaton. Strict emptiness is undecidable [19, 20]; the value cannot be approximated to within any fixed error smaller than $1/2$ [6, 16]; and whether $V(A) = 1$ is undecidable [12]. Fijalkow derives these results from one theorem [9]. They are statements about the map $A \mapsto V(A)$, and they are compatible with every single value being a computable real, in the same way that equality to 0 is undecidable for computable reals although each real in question is computable. The remark of Fijalkow, Gimbert, Kelmendi and Oualhadj that, as a consequence of Paz's undecidability result, "the value of an automaton is not computable in general" [10] refers to this map. Rote proved that strict emptiness is undecidable even for one fixed automaton whose initial distribution, or whose vector of final weights, is the input [22, 23]; the input there is not the threshold, and his theorem does not show that the value of one automaton is noncomputable. For channels, Boche, Schaefer and Poor treat a single computable finite-state channel with a noncomputable capacity as a stronger, hypothetical form of non-computability, and cite examples for compound and averaged channels with countably infinite sets of states [3, Remark 31]. For probabilistic finite automata, Theorem C below gives one automaton with a noncomputable value, and Theorem A shows that every left-c.e. real occurs; through the

channels of Elkouss and Pérez-García [8], the same holds for capacities of finite-state channels with rational transition probabilities (Corollary 9.7).

Theorem C. Fix an effective enumeration of Turing machines. Let H be the set of numbers n such that the n-th machine halts on input n, and let $\alpha_* = 1/4 + \sum_{n \in H} 4^{-(n+2)}$. Then α_* is not computable, and it is the value of the rational probabilistic finite automaton A_* of Section 6, built with the constants $a = 1/8$, $b = 1/2$ and $B = 16$.

In particular, for the one automaton A_*, the strict emptiness problem with the threshold as the only input is undecidable. Once an effective enumeration of Turing machines is fixed, the automaton A_* is obtained from a specified Turing machine by the compilation of Sections 3 to 5; its matrices are large, and we do not list them. Reals defined in this way from a computably enumerable undecidable set go back to Specker [25].

Corollary D. The halting probability of every prefix-free machine, in particular Chaitin's halting probability Ω of a universal prefix-free machine, is the value of a rational probabilistic finite automaton with two input letters and at most N states.

It is well known that every left-c.e. real in $[0, 1]$ is the halting probability of some prefix-free machine [4, 7], so Theorem A says that the values of rational probabilistic finite automata are exactly the halting probabilities of prefix-free machines (Remark 6.5).

The construction is effective once a rational interval $a < \alpha < b$ around the target is given, and it cannot be made effective from a left-c.e. index of the target alone: whether $V(A) = 0$ can be decided from A, while whether a left-c.e. real given by an approximating sequence is positive cannot (Proposition 8.1).

Theorem A has the shape of the theorem of Hochman and Meyerovitch that the topological entropies of shifts of finite type on $\mathbb{Z}^d$, $d \geq 2$, are exactly the nonnegative right-c.e. reals [13]. In both cases a quantity attached to a finite description has an evident one-sided approximation, from above for entropy and from below for the value of an automaton, and every real with such an approximation occurs. We use this only as an analogy; the methods are different.

1.1 How the automaton works

Fix a left-c.e. real $\alpha \in (0, 1)$ and rationals $0 < a < \alpha < b < 1$. For a binary word $s = s_1 \cdots s_k$ let $x(s) = \sum_{i=1}^{k} s_i 2^{i-k-1}$, the binary fraction $0.s_k s_{k-1} \cdots s_1$, and let $q(s) = a + (b - a)x(s)$. The set W_α of words s with $q(s) < \alpha$ is computably enumerable, so some counter program outputs exactly the words of W_α along its halting runs (Section 3). The input of the automaton is meant to be a sequence of *certificates*, each listing a run of this program: its instructions in order, and the counter values in unary (Figure 1). A deterministic part of the automaton checks everything except the counter arithmetic. The arithmetic, which a finite automaton cannot check, is tested at random. At the start of each certificate the automaton picks one of two tests with equal probability. The *joint* test survives each unary symbol of both sides of every claimed equation with probability $1/B$; the *split* test picks one side of each equation at random and survives each of its symbols with probability $1/B^2$. On a correct run the two tests survive with the same probability. If any equation is false, the split test is at least $\kappa_B = (B + B^{-1})/2$ times as likely to survive (Lemma 4.1 and Corollary 4.6). Independently, a two-state register holds $x(s)$, for the output s of the claimed run, as the probability of one of its states. At the end of a certificate, a surviving joint test accepts with probability $q(s)$, a surviving split test rejects with probability $1 - q(s)$, and all other probability starts afresh. Acceptance and rejection are permanent.

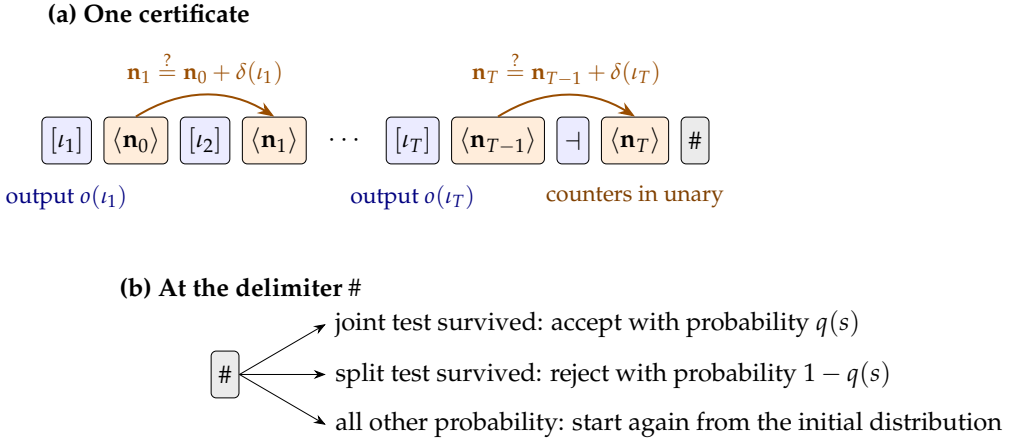


Figure 1: (a) A certificate lists a claimed run of a counter program. Each row is the next instruction, or $\dashv$ for the last row, followed by the counter values in unary. A deterministic parser checks everything except the equations between consecutive rows, which are tested at random. (b) At the delimiter a surviving joint test accepts with probability $q(s)$ and a surviving split test rejects with probability $1 - q(s)$, where s is the output listed in the certificate; everything else starts again. Acceptance and rejection are permanent.

Of the probability that a certificate settles, a correct certificate therefore accepts the fraction $q(s) < \alpha$, and an incorrect one a fraction at most $b/(b + \kappa_B(1 - b))$, which is below a once B is large. Every input word, however it mixes correct, incorrect and malformed certificates, is accepted with probability at most α. Repeating one correct certificate drives the acceptance probability up to $q(s)$, and these rewards approach α.

1.2 Prior work

Table 1 lists what was known about the value and what is proved here. The verification of counter-machine runs by a probabilistic finite automaton, through coin-flipping comparisons of unary lengths, is due to Condon and Lipton [6], who used an equality test of Freivalds [11] to prove that emptiness is undecidable with a gap; Madani, Hanks and Condon [15, 16] and Rote [22] give detailed accounts. Our joint and split tests are a form of this test. What we add is the payout at the delimiter: the surviving tests accept and reject in proportions set by a reward read from the certificate's output, so that the value of the automaton is the supremum of the rewards rather than one of two values separated by a gap. For partially observable Markov decision processes the value is likewise known to be uncomputable as a function of the process [16]; Proposition 9.5 records the range of values for reachability objectives.

Reading guide. Section 2 fixes definitions and proves the easy inclusion. Section 3 constructs counter programs that output a given computably enumerable set of words; it is Minsky's simulation of Turing machines with outputs added, and a reader who knows it can take Lemmas 3.2 and 3.3 as given. Section 4 builds the random check of one certificate, and Section 5 adds the payout and proves Theorem A. Section 6 proves Theorem C and Corollary D, the latter using Theorem B, which Section 7 proves; and Section 8 discusses uniformity. Section 9 collects open problems, with two classes of automata whose values are computable and the corresponding statement for partially observable Markov decision processes.

Statement	Scope	Source
Every value is left-c.e.	every automaton	folklore
Whether $V(A) > \lambda$ is undecidable	uniform in A, λ	[19, 20]
$V(A)$ cannot be approximated to within any fixed error below $1/2$	uniform in A	[6, 16, 9]
Whether $V(A) = 1$ is undecidable	uniform in A	[12]
Emptiness is undecidable for one fixed set of matrices, with the initial distribution or the final vector as input	one set of matrices, varying vectors	[22, 23]
One automaton has a noncomputable value	one automaton	Theorem C
The values are exactly the left-c.e. reals in $[0, 1]$	every left-c.e. real	Theorem A
Two letters and at most N states realize every value	one bound for all values	Theorem B
Every prefix-free halting probability is a value	every halting probability	Corollary D

Table 1: What was known about the value $V(A)$ of a rational probabilistic finite automaton, and what is proved here.

2 Preliminaries

Definition 2.1. A *rational probabilistic finite automaton* is a tuple $A = (Q, \Sigma, (M_\ell)_{\ell\in\Sigma}, \pi, F)$, where Q and Σ are finite nonempty sets, each M_ℓ is a row-stochastic $Q \times Q$ matrix with rational entries, π is a probability row vector on Q with rational entries, and $F \subseteq Q$. For a word $w = \ell_1 \cdots \ell_k$ over Σ let $M_w = M_{\ell_1} \cdots M_{\ell_k}$, with M_ε the identity, and let $P_A(w) = \pi M_w \mathbf{1}_F$, where $\mathbf{1}_F$ is the indicator column of F. The *value* of A is $V(A) = \sup_{w\in\Sigma^*} P_A(w)$.

The empty word is included, so $V(A) \geq \pi\mathbf{1}_F$. We describe automata by random transition rules: to say that on letter ℓ the automaton moves from state p to a random state with a given law means that row p of M_ℓ is that law. An automaton made of finitely many components, each with finitely many states, that update by independent random choices has the product of the component state sets as its state set, and the entries of its matrices are products of the probabilities of the components; it is a rational automaton when all these probabilities are rational.

Definition 2.2. A real α is *left-c.e.* if $\alpha = \lim_n \alpha_n$ for a computable nondecreasing sequence (α_n) of rationals. An index of a Turing machine that computes $n \mapsto \alpha_n$ is a *left-c.e. index* of α.

If (α_n) is such a sequence, a rational r satisfies $r < \alpha$ if and only if $\alpha_n > r$ for some n. Hence the set of rationals below α is computably enumerable, uniformly in a left-c.e. index of α. An *enumeration index* of a computably enumerable set W is an index, in a fixed effective enumeration of Turing machines, of a machine that halts on exactly the elements of W.

Proposition 2.3 (folklore)**.** *The value of every rational probabilistic finite automaton is a left-c.e. real in* $[0, 1]$.

Proof. Enumerate Σ^* as $w_0, w_1, \ldots$. The numbers $v_n = \max_{i\leq n} P_A(w_i)$ are rationals in $[0, 1]$, computable from n by exact arithmetic, nondecreasing, and they converge to $\sup_i P_A(w_i) = V(A)$. □

3 Counter programs that output words

Definition 3.1. A *counter program* P with d counters consists of a finite set $\mathcal{C}$ of control states with two distinct states start and halt, and a finite set $\mathcal{I}$ of instructions. Each instruction ι has a source $\mathrm{src}(\iota) \in \mathcal{C} \setminus \{\mathsf{halt}\}$, a destination $\mathrm{dst}(\iota) \in \mathcal{C}$, a guard $g(\iota) \in \{0, +, *\}^d$, an increment $\delta(\iota) \in \{-1, 0, 1\}^d$ with $g_j(\iota) = +$ whenever $\delta_j(\iota) = -1$, and an output $o(\iota) \in \{0, 1, \varepsilon\}$. A *configuration* is a pair $(c, \mathbf{n}) \in \mathcal{C} \times \mathbb{N}^d$. The instruction ι is *enabled* at $(c, \mathbf{n})$ if $\mathrm{src}(\iota) = c$ and, for every j, $n_j = 0$ when $g_j(\iota) = 0$ and $n_j > 0$ when $g_j(\iota) = +$; it then leads to $(\mathrm{dst}(\iota), \mathbf{n} + \delta(\iota))$. A *run* from $\mathbf{m} \in \mathbb{N}^d$ is a sequence $(c_0, \mathbf{n}_0), \iota_1, (c_1, \mathbf{n}_1), \ldots, \iota_T, (c_T, \mathbf{n}_T)$ with $(c_0, \mathbf{n}_0) = (\mathsf{start}, \mathbf{m})$ and $c_T = \mathsf{halt}$, in which each ι_i is enabled at $(c_{i-1}, \mathbf{n}_{i-1})$ and leads to $(c_i, \mathbf{n}_i)$. Its output is the binary word $o(\iota_1) \cdots o(\iota_T)$, where ε is the empty word. We write $L(P, \mathbf{m})$ for the set of outputs of runs from $\mathbf{m}$, and $L(P) = L(P, \mathbf{0})$.

The program may be nondeterministic: several instructions may be enabled at one configuration. We prove two lemmas. The first is Minsky's simulation of Turing machines by counter machines [17, 18], with a nondeterministic phase that writes the output; the second is its universal version.

Lemma 3.2. *For every computably enumerable set W of binary words there is a counter program P with three counters such that $L(P) = W$. The program can be computed from an enumeration index of W.*

Lemma 3.3. *There are a counter program U with four counters x_0, x_1, x_2, x_3 and a sequence $(W_e)_{e \in \mathbb{N}}$ of sets of binary words such that $L(U, (e, 0, 0, 0)) = W_e$ for every e, and such that for every computably enumerable set W of binary words some e, computable from an enumeration index of W, satisfies $W_e = W$.*

Turing machines. We use deterministic one-tape Turing machines M with a finite set K of states, a start state k_0, a halting state k_h, a finite tape alphabet Γ containing 0, 1 and the blank $\square$, and a transition function $\delta_M \colon (K \setminus \{k_h\}) \times \Gamma \to K \times \Gamma \times \{-1, 0, 1\}$. The cells of the tape are indexed by $\mathbb{Z}$. On input $y = y_0 \cdots y_{m-1}$ the machine starts in k_0 with y_i in cell i, blanks elsewhere, and the head on cell 0; it *halts* on y if it reaches k_h. A set of binary words is computably enumerable if and only if it is the set of binary words on which some such machine halts [24]. For a word y we write y^R for its reversal; the reversal of a computably enumerable set is computably enumerable.

Stacks in counters. Fix a machine M, let $h = |\Gamma| + 1$, and fix a bijection $\mathrm{code} \colon \Gamma \to \{1, \ldots, h-1\}$. A finite sequence $\gamma_1 \gamma_2 \cdots \gamma_r$ over Γ, read as a stack with top γ_1, is stored in a counter as

$$\ulcorner \gamma_1 \cdots \gamma_r \urcorner = \sum_{i=1}^{r} \mathrm{code}(\gamma_i)\, h^{i-1},$$

and the empty sequence is stored as 0. All base-h digits of a stored number are nonzero, so the number determines the sequence. A configuration of M in state k, with tape $\tau \colon \mathbb{Z} \to \Gamma$ and head on cell z, is *represented* by the control state $(k, \tau(z))$ together with counter values $x_1 = \ulcorner \tau(z-1) \cdots \tau(z-r) \urcorner$, $x_2 = \ulcorner \tau(z+1) \cdots \tau(z+r') \urcorner$ and $x_3 = 0$, for some $r, r' \geq 0$ such that every cell beyond those listed is blank. Different choices of r, r' represent the same configuration.

Lemma 3.4 (Stack operations). *Let $i \in \{1,2\}$ and $\gamma \in \Gamma$. There are finite sets of instructions without output, $\mathrm{PUSH}_i(\gamma)$ with an entry state and an exit state, and POP_i with an entry state and exit states $\mathrm{exit}_{\gamma'}$ for $\gamma' \in \Gamma$, with the following properties. At every control state of these sets other than the exits, exactly one instruction is enabled at each configuration. Started at the entry of $\mathrm{PUSH}_i(\gamma)$ with $x_3 = 0$ and x_i storing a sequence σ, the instructions lead to the exit with x_i storing $\gamma\sigma$, $x_3 = 0$ and the other counters unchanged. Started at the entry of POP_i with $x_3 = 0$ and x_i storing σ, they lead to $\mathrm{exit}_{\gamma'}$, where γ' is the top of σ, or $\Box$ if σ is empty, with x_i storing σ without its top, $x_3 = 0$ and the other counters unchanged.*

Proof. For $\mathrm{PUSH}_i(\gamma)$ use control states $\eta, \eta_1, \dots, \eta_{h-1}, \zeta, \zeta_1, \dots, \zeta_{\mathrm{code}(\gamma)}$, with entry η. At η: if $x_i > 0$, decrement x_i, increment x_3 and go to η_1; if $x_i = 0$, go to ζ. At η_l for $l < h-1$: increment x_3 and go to η_{l+1}; at η_{h-1}: increment x_3 and go to η. At ζ: if $x_3 > 0$, decrement x_3, increment x_i and stay at ζ; if $x_3 = 0$, go to ζ_1. At ζ_l for $l < \mathrm{code}(\gamma)$: increment x_i and go to ζ_{l+1}; at $\zeta_{\mathrm{code}(\gamma)}$: increment x_i and go to the exit. Each pass from η back to η moves one unit out of x_i and h units into x_3, so on reaching ζ we have $x_i = 0$ and $x_3 = h \cdot \ulcorner\sigma\urcorner$; the loop at ζ moves this back, and the chain $\zeta_1, \dots, \zeta_{\mathrm{code}(\gamma)}$ adds $\mathrm{code}(\gamma)$. The result is $h\ulcorner\sigma\urcorner + \mathrm{code}(\gamma) = \ulcorner\gamma\sigma\urcorner$.

For POP_i use control states $\eta_0, \dots, \eta_{h-1}$ and $\zeta_0, \dots, \zeta_{h-1}$, with entry η_0. At η_l: if $x_i > 0$ and $l < h-1$, decrement x_i and go to η_{l+1}; if $x_i > 0$ and $l = h-1$, decrement x_i, increment x_3 and go to η_0; if $x_i = 0$, go to ζ_l. At ζ_l: if $x_3 > 0$, decrement x_3, increment x_i and stay at ζ_l; if $x_3 = 0$, go to $\mathrm{exit}_{\gamma'}$ with $\mathrm{code}(\gamma') = l$ if $l \geq 1$, and to $\mathrm{exit}_\Box$ if $l = 0$. By induction, after X decrements of x_i the program is at $\eta_{X \bmod h}$ with $x_3 = \lfloor X/h \rfloor$. Let $X_0 = \ulcorner\sigma\urcorner$. If σ is empty, $X_0 = 0$ and the program goes from η_0 to ζ_0 to $\mathrm{exit}_\Box$ with all counters unchanged. Otherwise $X_0 = h\ulcorner\sigma'\urcorner + \mathrm{code}(\gamma')$, where γ' is the top of σ and σ' the rest, so the decrements end at η_l with $l = \mathrm{code}(\gamma') \geq 1$ and $x_3 = \ulcorner\sigma'\urcorner$, and the loop at ζ_l moves $\ulcorner\sigma'\urcorner$ back into x_i. In both sets the enabled instruction at each state is unique, because the alternatives have complementary guards. $\Box$

Proof of Lemma 3.2. Let M be a Turing machine that halts on exactly the binary words in $W^R = \{s^R : s \in W\}$. The program P has the counters x_1, x_2, x_3, the control states start, halt, the pairs $(k, \gamma) \in K \times \Gamma$, and the states of the copies of the stack operations used below, each copy with its own states. Its instructions are:

(G1) for $v \in \{0,1\}$, an instruction from start with output v to the entry of a copy of $\mathrm{PUSH}_2(v)$ whose exit is start;

(G2) an instruction from start to the entry of a copy of POP_2 whose exit exit_γ is (k_0, γ) for each γ;

(S) for $k \neq k_h$ and $\gamma \in \Gamma$ with $\delta_M(k, \gamma) = (k', \gamma', D)$: if $D = 0$, an instruction from (k, γ) to (k', γ'); if $D = +1$, an instruction from (k, γ) to a copy of $\mathrm{PUSH}_1(\gamma')$ whose exit is the entry of a copy of POP_2 with exits $(k', \cdot)$; if $D = -1$, the same with $\mathrm{PUSH}_2(\gamma')$ and POP_1;

(H) for each γ, an instruction from (k_h, γ) to halt.

Only the instructions (G1) have an output. Instructions outside the copies of the stack operations have guard $(*, *, *)$ and increment 0.

Consider a run from $\mathbf{0}$. At start the run uses (G1) some number $m \geq 0$ of times, with bits $s_1, \dots, s_m$, each followed by its copy of PUSH_2, and then (G2). By Lemma 3.4, before (G2) we have $x_1 = x_3 = 0$ and $x_2 = \ulcorner s_m s_{m-1} \cdots s_1 \urcorner$, and after the copy of POP_2 the control state is (k_0, γ) with $\gamma = s_m$, or $\gamma = \Box$ if $m = 0$, and $x_2 = \ulcorner s_{m-1} \cdots s_1 \urcorner$. This represents the start of M on the input $s_m \cdots s_1 = s^R$, where $s = s_1 \cdots s_m$. From then on exactly one

instruction is enabled at each configuration, and by Lemma 3.4 a pass through one instruction (S) and its copies maps a representation of a configuration of M to a representation of the next configuration: for $D = +1$, pushing γ' onto x_1 lists the cells to the left of the new head position, and popping x_2 yields the symbol under it, or a blank. Hence the run reaches halt if and only if M halts on s^R, that is, if and only if $s \in W$. Its output is s, because only (G1) has an output. Conversely, for each $s \in W$ the choices above give a run with output s. So $L(P) = W$. Every step is effective in M, and M is computable from an enumeration index of W. □

Proof of Lemma 3.3. Fix an effective enumeration $(M_e)_{e\in\mathbb{N}}$ of the Turing machines above. By the existence of a universal Turing machine, which goes back to Turing [26] (for the formulation with halting used here see, for example, [18, 24]), there is one Turing machine T_U, whose tape alphabet contains a symbol $\sharp$, such that for all $e \in \mathbb{N}$ and all binary words y, T_U halts on the input $1^e\sharp y$ if and only if M_e halts on y. Let $W_e = \{s : M_e \text{ halts on } s^R\}$. If W is computably enumerable, so is W^R, and any e such that M_e halts on exactly the binary words in W^R satisfies $W_e = W$; such an e is computable from an index of W.

The program U has the counters $x_0, \dots, x_3$, where x_0 is only used as described now, and the instructions (G1), (S) and (H) of the proof of Lemma 3.2, with T_U in place of M. Instead of (G2) it has an instruction from start to a copy of $\mathrm{PUSH}_2(\sharp)$ whose exit is a new control state init, and at init: if $x_0 > 0$, an instruction that decrements x_0 and goes to a copy of $\mathrm{PUSH}_2(1)$ whose exit is init; if $x_0 = 0$, an instruction to a copy of POP_2 with exits $(k_0, \cdot)$. A run from $(e, 0, 0, 0)$ with output $s = s_1 \cdots s_m$ reaches init with $x_2 = \ulcorner \sharp s_m \cdots s_1 \urcorner$, leaves it with $x_0 = 0$ and $x_2 = \ulcorner 1^e\sharp s_m \cdots s_1 \urcorner$, and after the copy of POP_2 represents the start of T_U on $1^e\sharp s^R$. As before, the run reaches halt if and only if T_U halts on $1^e\sharp s^R$, that is, if and only if $s \in W_e$. The instructions of U do not depend on e. □

4 Checking a run with two random tests

4.1 The equality test

Lemma 4.1. *Let $B \geq 2$ be an integer and $\kappa_B = (B + B^{-1})/2$. For $n, n' \in \mathbb{N}$ put*

$$\varphi_B(n, n') = B^{-(n+n')}, \qquad \psi_B(n, n') = \tfrac{1}{2}\big(B^{-2n} + B^{-2n'}\big).$$

Then $\psi_B(n, n') = \varphi_B(n, n')$ if $n = n'$, and $\psi_B(n, n') \geq \kappa_B\, \varphi_B(n, n')$ if $n \neq n'$.

Proof. The ratio is $\psi_B/\varphi_B = g(n - n')$ with $g(t) = (B^t + B^{-t})/2$. The function g is even, $g(0) = 1$, and for integers $t \geq 0$

$$g(t+1) - g(t) = \tfrac{1}{2}(B-1)\big(B^t - B^{-t-1}\big) > 0.$$

Hence $g(t) \geq g(1) = \kappa_B$ for every integer $t \neq 0$. □

A finite automaton realizes φ_B by surviving each unary symbol of both lengths with probability $1/B$, and ψ_B by choosing one of the two lengths with a fair coin and surviving each of its symbols with probability $1/B^2$. Since $\kappa_B \to \infty$, a single false equation makes the second experiment much more likely to survive than the first.

4.2 Certificates

Fix a counter program P with d counters. Its certificate alphabet is

$$\Sigma_P = \{[\iota] : \iota \in \mathcal{I}\} \cup \{\dashv, \mathtt{u}, \diamond, \#\}.$$

For $\mathbf{n} \in \mathbb{N}^d$ let $\langle \mathbf{n} \rangle = \mathtt{u}^{n_1} \diamond \mathtt{u}^{n_2} \diamond \cdots \mathtt{u}^{n_d} \diamond$, the *d blocks* of $\mathbf{n}$.

Definition 4.2. A *certificate* is a word

$$t = [\iota_1]\langle \mathbf{n}_0 \rangle [\iota_2]\langle \mathbf{n}_1 \rangle \cdots [\iota_T]\langle \mathbf{n}_{T-1} \rangle \dashv \langle \mathbf{n}_T \rangle \#$$

with $T \geq 0$, $\iota_i \in \mathcal{I}$ and $\mathbf{n}_i \in \mathbb{N}^d$. Its *rows* are the words $[\iota_{i+1}]\langle \mathbf{n}_i \rangle$ for $i < T$ and $\dashv \langle \mathbf{n}_T \rangle$; row i has the *head* $[\iota_{i+1}]$ or $\dashv$. Its *output* is $\mathrm{out}(t) = o(\iota_1) \cdots o(\iota_T)$. It is *well formed* if

(W1) $T \geq 1$, $\mathrm{src}(\iota_1) = \mathsf{start}$, $\mathrm{src}(\iota_{i+1}) = \mathrm{dst}(\iota_i)$ for $1 \leq i < T$, and $\mathrm{dst}(\iota_T) = \mathsf{halt}$;

(W2) $\mathbf{n}_0 = \mathbf{0}$;

(W3) for $0 \leq i < T$, the vector $\mathbf{n}_i$ satisfies the guard of ι_{i+1}.

Its *equations* are the pairs, for $0 \leq i < T$ and $1 \leq j \leq d$,

$$E_{i,j} = \big(n_{i,j} + \delta^+,\ n_{i+1,j} + \delta^-\big), \qquad \delta = \delta_j(\iota_{i+1}),$$

where $\delta^+ = \max(\delta, 0)$ and $\delta^- = \max(-\delta, 0)$; the two numbers are its *members*. A well-formed certificate is *valid* if the two members of every equation are equal.

The members of $E_{i,j}$ are equal if and only if $n_{i+1,j} = n_{i,j} + \delta_j(\iota_{i+1})$. To a run ρ from $\mathbf{0}$ as in Definition 3.1 we associate the certificate $\mathrm{cert}(\rho) = [\iota_1]\langle \mathbf{n}_0 \rangle \cdots [\iota_T]\langle \mathbf{n}_{T-1} \rangle \dashv \langle \mathbf{n}_T \rangle \#$.

Lemma 4.3. *A certificate is valid if and only if it is* $\mathrm{cert}(\rho)$ *for a run ρ of P from $\mathbf{0}$, and then its output is the output of ρ.*

Proof. Put $c_0 = \mathsf{start}$ and $c_i = \mathrm{dst}(\iota_i)$. Conditions (W1)–(W3) and the equalities $\mathbf{n}_{i+1} = \mathbf{n}_i + \delta(\iota_{i+1})$ say exactly that $(c_0, \mathbf{n}_0), \iota_1, \ldots, \iota_T, (c_T, \mathbf{n}_T)$ is a run from $\mathbf{0}$. The outputs agree by definition. □

4.3 The parser

Lemma 4.4. *There is a deterministic finite automaton over Σ_P, the* parser, *with an initial state $p_\circ$ and an absorbing error state p_{err}, such that for every word w the parser started in $p_\circ$ is in $p_\circ$ after reading w if and only if w is a concatenation of well-formed certificates, and it does not enter p_{err} while it reads a prefix of such a concatenation. While it reads a letter* $\mathtt{u}$ *or* $\diamond$*, its state determines the head of the current row, the index j of the current block, and whether the current row is row* 0*.*

Proof. A state of the parser outside $\{p_\circ, p_{\mathrm{err}}\}$ records the control state c that the next head must have as its source, the head of the current row, the index $j \in \{1, \ldots, d+1\}$ of the current block ($d+1$ once the row is complete), whether the current block has a symbol $\mathtt{u}$ yet, and whether the current row is row 0. In $p_\circ$ the expected source is start, no row has started, and the next row is row 0. On a head $[\iota]$ at the start of a row, the parser checks $\mathrm{src}(\iota) = c$, records ι and sets $c = \mathrm{dst}(\iota)$. On $\dashv$ at the start of a row it checks $c = \mathsf{halt}$, which excludes $T = 0$ because $\mathsf{start} \neq \mathsf{halt}$, and then replaces c by a mark that is the source of no instruction and is not halt; so after a row with head $\dashv$ is complete, the only allowed letter is #. On $\mathtt{u}$ it checks that the current row is not row 0 and that the guard of the head does not require the current

counter to be zero; for these checks the head $\dashv$ of a final row counts as having a guard with no requirements. On $\diamond$ it checks that the guard does not require the current counter to be positive when the block is empty, and moves to the next block. On # it checks that the current row is complete and has head $\dashv$, and returns to $p_\circ$. Every other letter, for instance u or $\diamond$ after the last block of a row, and every failed check, leads to p_{err}. These checks are exactly (W1)–(W3) and the format of Definition 4.2, and each is made at the letter that completes the condition checked, so a prefix of a concatenation of well-formed certificates passes all of them. □

4.4 The checking automaton

Fix P and an integer $B \geq 2$. The checking automaton has the following states. A *live* state is either (p, j, r) or (p, s, ξ, r), where $p \neq p_{\mathrm{err}}$ is a parser state, j and s mark the joint and the split test, $r \in \{0, 1\}$ is the *register*, and $\xi = (\xi_j^{\mathrm{in}}, \xi_j^{\mathrm{out}})_{j \leq d}$ with entries in $\{\emptyset, 1, 2\}$ records the choices of the split test. A *dead* state is $(p, \dagger)$ with $p \neq p_{\mathrm{err}}$. There are two further states, yes and no. The initial distribution $\pi_\circ$ gives probability $1/2$ to $(p_\circ, \mathsf{j}, 0)$ and $1/2$ to $(p_\circ, \mathsf{s}, \emptyset, 0)$, where $\emptyset$ denotes the vector with all entries $\emptyset$.

In a block j of row i, let $\chi_j \in \{0, 1, 2\}$ be the number of equations of which this block is a member: $\chi_j = [i \geq 1] + [\text{the head of row } i \text{ is not } \dashv]$. By Lemma 4.4 the parser state determines χ_j. For a letter $\ell \neq \#$, let p' be the parser state after ℓ, and let $o(\ell) = o(\iota)$ if $\ell = [\iota]$ and $o(\ell) = \varepsilon$ otherwise. The two-state register matrices are

$$R_0 = \begin{pmatrix} 1 & 0 \\ 1/2 & 1/2 \end{pmatrix}, \qquad R_1 = \begin{pmatrix} 1/2 & 1/2 \\ 0 & 1 \end{pmatrix}, \qquad R_\varepsilon = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix},$$

in the state order $0, 1$. On $\ell \neq \#$ the checking automaton acts as follows.

(C1) If $p' = p_{\mathrm{err}}$, every live or dead state moves to no.

(C2) Otherwise a dead state $(p, \dagger)$ moves to $(p', \dagger)$. A live state moves its parser state to p' and its register from r to r' with probability $R_{o(\ell)}(r, r')$, independently of everything else, and it survives with the probability given below; if it does not survive it moves to $(p', \dagger)$.

(C3) Joint test. On $[\iota]$ it survives with probability $B^{-\sum_j |\delta_j(\iota)|}$; on u in block j with probability $B^{-\chi_j}$; on other letters with probability 1.

(C4) Split test. On $[\iota]$ it draws $\xi_j^{\mathrm{out}} \in \{1, 2\}$ uniformly and independently for $j = 1, \ldots, d$, and survives with probability

$$\prod_{j=1}^{d} B^{-2\left(\delta_j^+ [\xi_j^{\mathrm{out}} = 1] + \delta_j^- [\xi_j^{\mathrm{out}} = 2]\right)}, \qquad \delta_j = \delta_j(\iota).$$

On u in block j it survives with probability $B^{-2([\xi_j^{\mathrm{in}} = 2] + [\xi_j^{\mathrm{out}} = 1])}$. On the separator $\diamond$ that closes block j it survives with probability 1 and replaces $(\xi_j^{\mathrm{in}}, \xi_j^{\mathrm{out}})$ by $(\xi_j^{\mathrm{out}}, \emptyset)$. On other letters it survives with probability 1 and keeps ξ.

Here $[\cdot]$ is 1 if the condition holds and 0 otherwise. The choice ξ_j^{out} drawn at the head of row i is the choice of the split test for the equation $E_{i,j}$: the value 1 selects its first member and 2 its second. It is kept in ξ_j^{out} while block j of row i is read, then in ξ_j^{in} while block j of row $i + 1$ is read, and then discarded. At each moment at most two equations per counter have a recorded choice, so the state set is finite. All probabilities are rational, and each row sums to 1. The rows of yes and no, and the action of #, are given in Section 5.

For a well-formed certificate $t = t'\#$ with equations $\mathcal{E}(t) = \{E_{i,j}\}$, put

$$\Phi(t) = \frac{1}{2} \prod_{(n,n') \in \mathcal{E}(t)} \varphi_B(n, n'), \qquad \Psi(t) = \frac{1}{2} \prod_{(n,n') \in \mathcal{E}(t)} \psi_B(n, n').$$

For a binary word $s = s_1 \cdots s_k$ let $x(s) = \sum_{i=1}^{k} s_i 2^{i-k-1}$, so that $x(\varepsilon) = 0$ and $x(sv) = (x(s) + v)/2$ for a bit v.

Lemma 4.5. *Let $t = t'\#$ be a well-formed certificate with output s. Started from $\pi_\circ$, after reading t' the checking automaton is in live joint states with total probability $\Phi(t)$, in live split states with total probability $\Psi(t)$, and in dead states otherwise. Within each of the two live parts, the register is* 1 *with conditional probability $x(s)$.*

Proof. No letter of t' leads the parser to p_{err}, so (C1) does not occur, and the parser state is the same for all states at every moment.

Joint test. Its survival probability is the product of the factors in (C3). A symbol $\mathtt{u}$ in block j of row i contributes B^{-1} for each equation of which the block is a member, and the head $[\iota_{i+1}]$ contributes B^{-1} for each unit of the offsets δ^+ and δ^- in the equations $E_{i,j}$, since $\delta^+ + \delta^- = |\delta|$. The product is therefore $\prod_{(n,n') \in \mathcal{E}(t)} B^{-(n+n')}$, and with the initial factor $1/2$ the probability is $\Phi(t)$.

Split test. Fix the choices $\xi_{i,j} \in \{1, 2\}$ for all equations. Given these, the survival probability is the product of the factors in (C4). By the description of the slots above, a symbol $\mathtt{u}$ in block j of row i contributes B^{-2} if $\xi_{i-1,j} = 2$ (it lies in the second member of $E_{i-1,j}$, which was chosen) and B^{-2} if $\xi_{i,j} = 1$ (it lies in the first member of $E_{i,j}$, which was chosen); the head of row i contributes $B^{-2\delta^+}$ if $\xi_{i,j} = 1$ and $B^{-2\delta^-}$ if $\xi_{i,j} = 2$, for each j. Collecting the factors by equation, the survival probability given the choices is $\prod_{i,j} f_{i,j}(\xi_{i,j})$, where $f_{i,j}(1) = B^{-2m}$ and $f_{i,j}(2) = B^{-2m'}$ for $E_{i,j} = (m, m')$. The choices are independent and uniform, so averaging gives $\prod_{i,j} \frac{1}{2}(f_{i,j}(1) + f_{i,j}(2)) = \prod_{(m,m') \in \mathcal{E}(t)} \psi_B(m, m')$; with the initial factor $1/2$ the probability is $\Psi(t)$.

Register. The register moves by $R_{o(\ell)}$ independently of the other components, and its matrices depend only on the letter, so the joint law of the register and the other components remains a product law. If the register is 1 with probability x, then after R_0 it is 1 with probability $x/2$ and after R_1 with probability $(1 - x)/2 + x = (1 + x)/2$. Starting from 0, after the heads of t' it is 1 with probability $x(s)$. □

Corollary 4.6. *Let t be a well-formed certificate. Then $\Phi(t) > 0$. If t is valid, $\Psi(t) = \Phi(t)$. If t is not valid, $\Psi(t) \geq \kappa_B \Phi(t)$.*

Proof. All factors are positive. By Lemma 4.1, every factor ratio ψ_B/φ_B is at least 1, equal to 1 for an equation with equal members, and at least κ_B for an equation with unequal members. A certificate that is well formed but not valid has at least one such equation. □

5 The payout and the proof of Theorem A

Fix rationals $0 < a < b < 1$ and put $\beta_0 = a$ and $\beta_1 = b$. For a binary word s let $q(s) = a + (b - a)x(s)$, so $q(s) \in [a, b)$. The automaton $A(P, a, b, B)$ consists of the checking automaton, with initial distribution $\pi_\circ$ and accepting set $F = \{\mathsf{yes}\}$, and the following transitions.

(P1) The states yes and no are absorbing under every letter.

(P2) On #, if the parser moves to p_{err}, every live or dead state moves to no.

(P3) On #, if the parser moves to $p_\circ$, that is, at the end of a well-formed certificate: a live joint state with register r moves to yes with probability β_r; a live split state with register r moves to no with probability $1-\beta_r$; all remaining probability of a live state, and all probability of a dead state, is distributed according to $\pi_\circ$.

The rows in (P3) are rational combinations of a point mass and $\pi_\circ$, so $A(P,a,b,B)$ is a rational probabilistic finite automaton. The only accepting state is yes, which is entered only in (P3).

Lemma 5.1. *Let t be a well-formed certificate with output s. Started from $\pi_\circ$, after reading t the automaton is in* yes *with probability* $\operatorname{acc}(t) = \Phi(t)q(s)$, *in* no *with probability* $\operatorname{rej}(t) = \Psi(t)(1-q(s))$, *and the remaining probability* $1-\operatorname{acc}(t)-\operatorname{rej}(t)$ *is distributed according to* $\pi_\circ$.

Proof. By Lemma 4.5, before the final # the live joint states carry probability $\Phi(t)$, with the register 1 with conditional probability $x(s)$. By (P3) they move to yes with probability $\Phi(t)\big((1-x(s))a + x(s)b\big) = \Phi(t)q(s)$. In the same way the live split states move to no with probability $\Psi(t)\big((1-x(s))(1-a) + x(s)(1-b)\big) = \Psi(t)(1-q(s))$. Everything else is redistributed according to $\pi_\circ$. □

Lemma 5.2. *Let $\theta \in [0,1]$, and suppose that* $\operatorname{acc}(t) \le \theta\,(\operatorname{acc}(t)+\operatorname{rej}(t))$ *for every well-formed certificate t. Then $P_A(w) \le \theta$ for every word $w \in \Sigma_P^*$.*

Proof. A well-formed certificate contains exactly one #, at its end, so every word can be written uniquely as $w = t_1 \cdots t_m z$, where $t_1, \dots, t_m$ are well-formed certificates and no prefix of z is a well-formed certificate. Put $\mu_1 = 1$ and $\mu_{i+1} = \mu_i(1-\operatorname{acc}(t_i)-\operatorname{rej}(t_i))$. By Lemma 5.1 and linearity, after reading $t_1 \cdots t_i$ the automaton is in yes with probability $\sum_{l \le i} \mu_l \operatorname{acc}(t_l)$, and the probability outside $\{\text{yes}, \text{no}\}$ equals μ_{i+1} and is distributed according to $\pi_\circ$. While z is read, the parser starts in $p_\circ$, and a transition into yes would require it to complete a well-formed certificate, which does not happen; the probability in yes does not decrease, by (P1). Hence

$$P_A(w) = \sum_{i=1}^{m} \mu_i \operatorname{acc}(t_i) \le \theta \sum_{i=1}^{m} \mu_i\big(\operatorname{acc}(t_i)+\operatorname{rej}(t_i)\big) = \theta \sum_{i=1}^{m} (\mu_i - \mu_{i+1}) = \theta(1-\mu_{m+1}) \le \theta. \quad \square$$

Lemma 5.3. *Let t be a valid certificate with output s. Then $P_A(t^m) = q(s)\big(1-(1-\Phi(t))^m\big)$ for every $m \ge 1$.*

Proof. By Corollary 4.6, $\Psi(t) = \Phi(t)$, so $\operatorname{acc}(t)+\operatorname{rej}(t) = \Phi(t)$. By Lemma 5.1, as in the proof of Lemma 5.2, $P_A(t^m) = \sum_{i=1}^{m} (1-\Phi(t))^{i-1}\Phi(t)q(s) = q(s)\big(1-(1-\Phi(t))^m\big)$. □

Theorem 5.4. *Let $\alpha \in (0,1)$ be left-c.e., let a, b be rationals with $0 < a < \alpha < b < 1$, and let $B \ge 2$ be an integer with*

$$\kappa_B > \frac{b(1-a)}{a(1-b)}. \tag{1}$$

Let P be a counter program with $L(P) = W_\alpha$, where $W_\alpha = \{s \in \{0,1\}^* : q(s) < \alpha\}$. Then $V(A(P,a,b,B)) = \alpha$. Consequently Theorem A holds.

Proof. The set W_α is computably enumerable: given s, compute the rational $q(s)$ and search for n with $\alpha_n > q(s)$, where (α_n) is a computable nondecreasing sequence of rationals converging to α. So P exists by Lemma 3.2, and B exists because $\kappa_B \to \infty$. Write $A = A(P,a,b,B)$.

Upper bound. Let t be a well-formed certificate with output s. By Corollary 4.6, $\Phi(t) > 0$ and $\operatorname{acc}(t)+\operatorname{rej}(t) \ge \Phi(t)q(s) > 0$. If t is valid, then $s \in L(P) = W_\alpha$ by Lemma 4.3,

and $\mathrm{acc}(t)/(\mathrm{acc}(t)+\mathrm{rej}(t)) = q(s) < \alpha$. If t is not valid, then $\Psi(t) \geq \kappa_B \Phi(t)$, and since $f(q) = q/(q+\kappa_B(1-q))$ is increasing on $[0,1]$ and $q(s) \leq b$,

$$\frac{\mathrm{acc}(t)}{\mathrm{acc}(t)+\mathrm{rej}(t)} \leq \frac{q(s)}{q(s)+\kappa_B(1-q(s))} \leq \frac{b}{b+\kappa_B(1-b)} < a < \alpha;$$

the strict inequality is a rearrangement of (1). By Lemma 5.2 with $\theta = \alpha$, $P_A(w) \leq \alpha$ for every word w.

Lower bound. Let $s \in W_\alpha$. By Lemma 4.3 some valid certificate t has output s, and by Lemma 5.3 $V(A) \geq \sup_m P_A(t^m) = q(s)$. As s ranges over binary words, $x(s)$ ranges over all dyadic rationals in $[0,1)$. The number $\vartheta = (\alpha - a)/(b-a)$ lies in $(0,1)$, and $q(s) < \alpha$ if and only if $x(s) < \vartheta$. Dyadic rationals below ϑ come arbitrarily close to ϑ, so $\sup_{s \in W_\alpha} q(s) = a + (b-a)\vartheta = \alpha$, and $V(A) \geq \alpha$.

Theorem A. Every value is left-c.e. in $[0,1]$ by Proposition 2.3. Every left-c.e. $\alpha \in (0,1)$ has rationals a, b as above and is a value by the first part. The values 0 and 1 are those of the one-state automata with $F = \emptyset$ and $F = Q$. □

6 An explicit noncomputable value and halting probabilities

Let (M_n) be the fixed effective enumeration of Turing machines of Lemma 3.3, let H be the set of n such that M_n halts on input n, written in binary, and put $\omega_n = 4^{-(n+2)}$ and $\alpha_* = 1/4 + \sum_{n \in H} \omega_n$. The set H is computably enumerable and, by the diagonal argument, not computable (see [24]).

Lemma 6.1. *Let $\alpha_t = 1/4 + \sum\{\omega_n : n \leq t,\ M_n$ halts on input n within t steps$\}$. The sequence (α_t) is computable, nondecreasing and converges to α_*, and $1/4 \leq \alpha_* \leq 1/3$. The real α_* is not computable.*

Proof. The first assertions are clear, and $\sum_{n \geq 0} \omega_n = (1/16)/(1-1/4) = 1/12$. For every n, $\sum_{l>n} \omega_l = \omega_n/3$. Suppose that rational approximations to α_* with any prescribed error were computable. We decide whether $n \in H$ by induction on n. Given $H \cap \{0,\dots,n-1\}$, the residual $\alpha_* - 1/4 - \sum_{l<n,\, l \in H} \omega_l$ is at most $\omega_n/3$ if $n \notin H$ and at least ω_n if $n \in H$. Subtract the same rational from an approximation of α_* with error less than $\omega_n/12$: the result is at most $5\omega_n/12$ in the first case and at least $11\omega_n/12$ in the second, and comparing it with $\omega_n/2$ decides whether $n \in H$. This would make H computable. □

Let T_* be the Turing machine that, on a binary input y, computes the rational $q = 1/8 + (3/8) \cdot 0.y$, where $0.y$ is the binary fraction with the digits of y in order, and then computes $\alpha_0, \alpha_1, \dots$ and halts as soon as $\alpha_t > q$. Since $x(s) = 0.s^R$ for every binary word s, the machine T_* halts on s^R if and only if $1/8 + (3/8)x(s) < \alpha_*$. Let P_* be the program of Lemma 3.2 built from $M = T_*$, so that $L(P_*) = \{s : 1/8 + (3/8)x(s) < \alpha_*\}$, and let $A_* = A(P_*, 1/8, 1/2, 16)$.

Theorem 6.2. *$V(A_*) = \alpha_*$, and α_* is not computable. In particular Theorem C holds.*

Proof. We check the hypotheses of Theorem 5.4 with $a = 1/8$, $b = 1/2$ and $B = 16$. By Lemma 6.1, α_* is left-c.e. and $a < 1/4 \leq \alpha_* \leq 1/3 < b$. Here $b(1-a)/(a(1-b)) = 7$ and $\kappa_{16} = 257/32 > 7$; correspondingly $b/(b + \kappa_{16}(1-b)) = 32/289 < 1/8$. With these constants $q(s) = 1/8 + (3/8)x(s)$ and $L(P_*) = W_{\alpha_*}$. Theorem 5.4 gives $V(A_*) = \alpha_*$, and Lemma 6.1 shows that α_* is not computable. □

For each fixed real α, α is computable if and only if its lower cut $\{\lambda \in \mathbb{Q} : \lambda < \alpha\}$ is decidable; the equivalence is not uniform in an index of α. Since $V(A_*) > \lambda$ if and only if $P_{A_*}(w) > \lambda$ for some word w, the strict emptiness problem for the fixed automaton A_*, with the rational threshold λ as its only input, is undecidable.

A *prefix-free machine* is a partial computable function $\mathcal{M}$ from binary words to binary words whose domain is prefix free; its halting probability is $\Omega_{\mathcal{M}} = \sum_{\sigma \in \operatorname{dom} \mathcal{M}} 2^{-|\sigma|}$.

Corollary 6.3. *For every prefix-free machine $\mathcal{M}$ there is a rational probabilistic finite automaton with two input letters and at most N states whose value is $\Omega_{\mathcal{M}}$, where N is the number of Theorem B. In particular Corollary D holds.*

Proof. The words σ in the domain correspond to the intervals $[0.\sigma, 0.\sigma + 2^{-|\sigma|})$ of length $2^{-|\sigma|}$, which are pairwise disjoint because no word of the domain is a prefix of another. Hence every finite partial sum of $\Omega_{\mathcal{M}}$ is at most 1. Enumerating the domain and adding $2^{-|\sigma|}$ for each new word gives a computable nondecreasing sequence of rationals converging to $\Omega_{\mathcal{M}}$, so $\Omega_{\mathcal{M}}$ is a left-c.e. real in $[0, 1]$, and Theorem B applies. For a universal prefix-free machine, $\Omega_{\mathcal{M}}$ is Chaitin's halting probability [5]. □

Remark 6.4. By results of Chaitin [5], of Calude, Hertling, Khoussainov and Wang [4], and of Kučera and Slaman [14] (see [7]), the left-c.e. reals that are Martin-Löf random are exactly the halting probabilities of universal prefix-free machines. With Theorem A, the Martin-Löf random values of rational probabilistic finite automata are exactly these numbers.

Remark 6.5. Conversely, it is well known that every left-c.e. $\alpha \in [0, 1]$ is $\Omega_{\mathcal{M}}$ for some prefix-free machine $\mathcal{M}$ [4, 7]; we recall the argument. Let (α_n) be a computable nondecreasing sequence of rationals with limit α; replacing α_n by $\max(0, \alpha_{n-1})$ for $n \geq 1$ and setting $\alpha_0 = 0$, we may assume $\alpha_0 = 0$ and $\alpha_n \geq 0$. The dyadic rationals $\lfloor 2^n \alpha_n \rfloor / 2^n$ also form a nondecreasing sequence with limit α; their increments are finite sums of powers 2^{-k} with total $\alpha \leq 1$, and the Kraft–Chaitin theorem [5, 7] gives a prefix-free machine whose domain has one word of length k for each such power. So Theorem A says that the values of rational probabilistic finite automata are exactly the halting probabilities of prefix-free machines.

7 Two letters and a fixed number of states

Lemma 7.1. *Let A be a rational probabilistic finite automaton with n states and $m \geq 1$ letters, and let $l \geq 1$ satisfy $2^l \geq m$. There is a rational probabilistic finite automaton A' over $\{0, 1\}$ with $n(2^l - 1) + 1$ states and $V(A') = V(A)$.*

Proof. Fix distinct codes $\operatorname{code}(\ell) \in \{0, 1\}^l$ for the letters of A. The states of A' are the pairs (p, σ) of a state p of A and a binary word σ of length less than l, and a rejecting sink. The initial distribution gives (p, ε) the probability $\pi(p)$, and (p, σ) is accepting if and only if $p \in F$. On a bit y, the state (p, σ) moves to $(p, \sigma y)$ if $|\sigma y| < l$; to (p', ε) with probability $M_\ell(p, p')$ if $\sigma y = \operatorname{code}(\ell)$; and to the sink if σy has length l and is not a code. The sink is absorbing. A binary word that contains no invalid code is a concatenation $\operatorname{code}(\ell_1) \cdots \operatorname{code}(\ell_k)\sigma$ with $|\sigma| < l$, and it is accepted by A' with probability $P_A(\ell_1 \cdots \ell_k)$, because the accepting set is read from the first component in every phase. A word that contains an invalid code is accepted with probability 0. Hence $V(A') \leq V(A)$, and $P_{A'}(\operatorname{code}(\ell_1) \cdots \operatorname{code}(\ell_k)) = P_A(\ell_1 \cdots \ell_k)$ gives $V(A') \geq V(A)$. □

The index check. Let U be the program of Lemma 3.3, with $d = 4$ counters indexed $0, \dots, 3$, and fix $e \in \mathbb{N}$. We modify the construction of Sections 4 and 5 so that it checks runs of U from $(e, 0, 0, 0)$ without storing e in the state.

(I1) Condition (W2) becomes $n_{0,1} = n_{0,2} = n_{0,3} = 0$, with $n_{0,0}$ arbitrary; the parser allows symbols $\mathfrak{u}$ in block 0 of row 0.

(I2) Every certificate has one more equation $E_* = (e, n_{0,0})$, whose first member is the number e itself.

(I3) In block 0 of row 0, $\chi_0 = 1 + [\text{the head of row 0 is not } \dashv]$. The choice of the split test for E_* is kept in ξ_0^{in} during row 0.

(I4) A new initial distribution $\pi_\circ^e$ replaces $\pi_\circ$ everywhere, including in (P3). It gives the probability $B^{-e}/2$ to $(p_\circ, \mathsf{j}, 0)$, the probability $1/4$ to the state $(p_\circ, \mathsf{s}, \xi, 0)$ with $\xi_0^{\text{in}} = 2$ and all other entries $\emptyset$, the probability $B^{-2e}/4$ to the same state with $\xi_0^{\text{in}} = 1$, and the remaining probability, $(1 - B^{-e})/2 + (1 - B^{-2e})/4$, to the dead state $(p_\circ, \dagger)$.

In the joint test, (I4) contributes the factor B^{-e} and (I3) the factor B^{-1} for each symbol of block 0 of row 0 on account of E_*, so E_* contributes $\varphi_B(e, n_{0,0})$. In the split test, choice 1 contributes B^{-2e} through (I4) and nothing in block 0, and choice 2 contributes B^{-2} for each symbol of that block, so E_* contributes $\psi_B(e, n_{0,0})$. Write $A_U(e, a, b, B)$ for the resulting automaton.

Lemma 7.2. *With these modifications, Lemma 4.3 holds with runs from $(e, 0, 0, 0)$ in place of runs from $\mathbf{0}$, where valid means that both members of every equation, E_* included, are equal; and Lemmas 4.5 and 5.1 to 5.3 and Corollary 4.6 hold for $A_U(e, a, b, B)$ with $\pi_\circ^e$ in place of $\pi_\circ$ and E_* included in $\mathcal{E}(t)$. The states and letters of $A_U(e, a, b, B)$ do not depend on e, a, b or B.*

Proof. The equation E_* holds if and only if $n_{0,0} = e$; together with the modified (W2) this says that the certificate starts at $(\mathsf{start}, (e, 0, 0, 0))$, and the rest of the proof of Lemma 4.3 is unchanged. The parser modified by (I1) satisfies Lemma 4.4 with the modified (W2), by the same proof; in particular, after a row with head $\dashv$ only # is allowed. In the proof of Lemma 4.5 the factors contributed by E_* were computed above, and the choice for E_* is independent of all other choices; the other factors are as before. The proofs of Corollary 4.6 and Lemmas 5.1 to 5.3 use only Lemmas 4.1 and 4.5, the fact that the probability that is not absorbed is redistributed according to the initial distribution, and the parser, and they carry over. The parser depends only on U; the state set consists of parser states, the marks $\mathsf{j}, \mathsf{s}, \dagger$, the slot vectors ξ for four counters, the register and the two absorbing states; the alphabet is Σ_U. The parameters e, a, b, B enter only the transition probabilities and the initial distribution. □

Theorem 7.3. *Let N_0 be the number of states of $A_U(e, a, b, B)$, which does not depend on e, a, b, B by Lemma 7.2, let l be the least integer with $2^l \geq |\Sigma_U|$, and let $N = N_0(2^l - 1) + 1$. Every left-c.e. real in $[0, 1]$ is the value of a rational probabilistic finite automaton over $\{0, 1\}$ with at most N states. In particular Theorem B holds.*

Proof. The values 0 and 1 are realized by one-state automata over $\{0, 1\}$. Let $\alpha \in (0, 1)$ be left-c.e., choose a, b, B as in Theorem 5.4, and choose e with $W_e = W_\alpha$ by Lemma 3.3. By Lemma 7.2, the proof of Theorem 5.4 applies verbatim to $A_U(e, a, b, B)$, with $L(U, (e, 0, 0, 0)) = W_e = W_\alpha$ in place of $L(P)$; so its value is α. Lemma 7.1 turns it into an automaton over $\{0, 1\}$ with N states and the same value. □

8 Uniformity

Proposition 8.1.

(i) Whether $V(A) = 0$ is decidable for rational probabilistic finite automata A.

(ii) There is no algorithm that, given a left-c.e. index of a real $\alpha \in [0,1]$, outputs a rational probabilistic finite automaton with value α. This remains true for indices of sequences whose limits lie in $\{0, 1/2\}$.

(iii) There is an algorithm that, given a left-c.e. index of a real α and rationals a, b with $0 < a < \alpha < b < 1$, outputs a rational probabilistic finite automaton over $\{0,1\}$ with at most N states and value α.

Proof. (i) $P_A(w) > 0$ if and only if some path of states, starting in the support of π and ending in F, has positive probability under w. So $V(A) > 0$ if and only if F is reachable from the support of π in the directed graph with an edge $p \to p'$ whenever $M_\ell(p,p') > 0$ for some letter ℓ, which is decidable.

(ii) For each n, the sequence $\alpha_t^{(n)}$ equal to $1/2$ if the Turing machine M_n of Section 6 halts on input n within t steps, and to 0 otherwise, is computable uniformly in n, nondecreasing, and has limit $1/2$ if $n \in H$ and 0 otherwise. An algorithm as in (ii), followed by the test of (i), would decide H.

(iii) From a and b compute the least B with (1). From the index of α and from a, b compute an enumeration index of W_α, then e with $W_e = W_\alpha$ (Lemma 3.3), and output the automaton of Theorem 7.3. □

The map from automata to left-c.e. indices of their values is computable (Proposition 2.3). By (ii) and (iii), a left-c.e. index of the target does not suffice to compute an automaton, while an index together with a rational interval (a,b) does. The obstruction in (ii) concerns the lower end; we do not know whether an index together with a rational $a \in (0,\alpha)$ alone suffices.

9 Open problems

Small automata. The automata of Theorem B have a fixed but large number of states. Few states are not enough.

Proposition 9.1. *The value of a rational probabilistic finite automaton with at most two states is rational.*

Proof. With one state the value is 0 or 1. With states $0, 1$, let x be the probability of state 1. Each letter acts by an affine map $f_\ell(x) = M_\ell(0,1) + (M_\ell(1,1) - M_\ell(0,1))x$ with rational coefficients, mapping $[0,1]$ into itself. Let O be the set of the numbers $f_w(x_0)$ over all words w, where x_0 is the initial probability of state 1, and let $x_- = \inf O$ and $x_+ = \sup O$. Since the maps f_ℓ are affine and continuous and $f_\ell(O) \subseteq O$, the interval $[x_-, x_+]$ is invariant under every f_ℓ; and every closed interval that contains x_0 and is invariant under every f_ℓ contains O, hence $[x_-, x_+]$. An interval $[u,v]$ is invariant under an affine map f if and only if $f(u), f(v) \in [u,v]$. So the pairs (u,v) with $0 \le u \le x_0 \le v \le 1$ and $u \le f_\ell(u), f_\ell(v) \le v$ for all ℓ form a rational polytope that contains (x_-, x_+) and on which $v \ge x_+$ and $u \le x_-$. The minimum of v on this polytope is x_+, and it is attained at a vertex, which is rational; likewise x_- is rational. The value is x_+, $1 - x_-$, 0 or 1, according to the accepting set. □

Question 9.2. Is every left-c.e. real in $[0,1]$ the value of a rational probabilistic finite automaton with three states and two letters? Less ambitiously, give an explicit small number of states, or an explicit automaton with few states and a noncomputable value.

Fixed matrices. For $\alpha \in (1/8, 1/2)$ the constants $a = 1/8$, $b = 1/2$ and $B = 16$ can be kept fixed in Theorem B, and then e enters only through the initial distribution $\pi^e_\circ$, which is also used in the rows (P3). Rote's fixed automaton has the initial distribution as its only input [23]. This suggests the following question.

Question 9.3. Are there finitely many fixed rational stochastic matrices and a fixed accepting set such that, as the rational initial distribution varies, the values include every left-c.e. real in some nondegenerate interval?

Noise. The construction uses transitions of probability 0: acceptance and rejection are permanent, and a failed test stays dead until the next delimiter. Some transitions of probability 0 are needed for a noncomputable value.

Proposition 9.4. *If all entries of all matrices M_ℓ are positive, then $V(A)$ is computable from A.*

Proof. Let $n = |Q|$, let $m_0 > 0$ be the least entry of all M_ℓ, and put $\eta = nm_0 \le 1$. Write $M_\ell = \eta J + (1-\eta)S_\ell$, where J has all entries $1/n$; if $\eta < 1$, each S_ℓ is stochastic, and if $\eta = 1$, every M_ℓ equals J. For probability vectors ν, ν' the difference $\nu - \nu'$ has coordinate sum 0, so $(\nu - \nu')J = 0$ and $\|(\nu - \nu')M_\ell\|_1 \le (1-\eta)\|\nu - \nu'\|_1$. For a word $w = w'v$ with $|v| = k$, applying this to $\nu = \pi M_{w'}$ and $\nu' = \pi$ gives $|P_A(w) - P_A(v)| \le \frac{1}{2}\|(\nu - \nu')M_v\|_1 \le (1-\eta)^k$. Hence $V(A)$ lies between the maximum of P_A over words of length at most k and that maximum plus $(1-\eta)^k$, for every k. □

Other finite-state models. A partially observable Markov decision process with finitely many states, actions and observations, rational transition probabilities, a rational initial distribution and a deterministic observation map, together with a target set, has as its value the supremum, over all policies that may use the history of actions and observations and may randomize, of the probability of ever visiting the target.

Proposition 9.5. *The values of such processes with reachability objectives are exactly the left-c.e. reals in $[0,1]$.*

Proof. Let V_k be the supremum over policies of the probability of visiting the target within k steps. For a fixed horizon, this probability is affine in each of the distributions that a policy chooses after the histories of length less than k, separately, since each history occurs at most once along a trajectory. Replacing these distributions one at a time by suitable single actions therefore does not decrease it, so V_k is attained by one of finitely many deterministic policies and is a rational number computable from k. The sequence V_k is nondecreasing, V_k is at most the value, and for every policy the probability of ever visiting the target is the limit of the probabilities of visiting it within k steps, hence at most $\sup_k V_k$. So the value is $\lim_k V_k$, a left-c.e. real in $[0,1]$. Conversely, let $\alpha \in (0,1)$ be left-c.e. and let A be an automaton with value α from Theorem 5.4; view it as a process with one observation, actions Σ_P and target $\{\mathsf{yes}\}$. Since yes is absorbing, the probability of visiting yes within k steps under a sequence of actions is the acceptance probability of its first k letters, so every policy visits yes with probability at most α, and the policies that play t^m show that the value is α. The values 0 and 1 are trivial. □

A probabilistic Büchi automaton [1] accepts an infinite word with the probability that the run visits the accepting set infinitely often. For the automata of Theorem 5.4, whose only accepting state is absorbing, this is the probability of ever reaching yes, so the supremum of these probabilities over infinite words is again α. Thus every left-c.e. real in $[0,1]$ is such a supremum for a probabilistic Büchi automaton.

Question 9.6. Which reals are suprema of the acceptance probabilities of rational probabilistic Büchi automata? Are they all left-c.e.?

Elkouss and Pérez-García showed, through probabilistic automata, that the capacity of a finite-state channel cannot be computed from the channel to within a fixed precision [8], and Boche, Schaefer and Poor proved further non-computability results of this uniform kind [3]. They treat a single computable channel with a noncomputable capacity as a stronger form of non-computability [3, Remark 31]. Such channels exist with finitely many states.

Corollary 9.7. *Every left-c.e. real in $[0,1]$ is the capacity of a finite-state channel with rational transition probabilities. In particular, some finite-state channel with rational transition probabilities has a capacity that is not a computable real.*

Proof. Elkouss and Pérez-García map each probabilistic finite automaton A that starts in a fixed state to a finite-state channel V_A with rational transition probabilities, in which one input register controls A, with identity and reset controls added, and they prove that the capacity of V_A equals the value of A [8, Theorem 1]. An automaton with a rational initial distribution π is converted to one with a fixed start state without changing its value: add a start state s, a nonaccepting absorbing sink and a new letter that sends s to π and every other state to the sink, and let each old letter act as before on the old states and send s to the sink. Only words that begin with the new letter can be accepted, and the probability of accepting such a word is the probability that the old automaton accepts the rest of it. Now apply Theorem 5.4. Theorem C gives a channel with capacity α_*, and Corollary D gives channels whose capacities are halting probabilities. □

The corollary is an inclusion; we do not determine which reals are capacities of finite-state channels with rational transition probabilities.

For measure-once quantum finite automata with rational or real algebraic amplitudes, Blondel, Jeandel, Koiran and Portier proved that it is decidable whether some word is accepted with probability strictly greater than a given rational threshold [2]. For the automata to which their theorem applies, the lower cut of the value is decidable, so the value is a computable real. The phenomenon of Theorem A does not occur there.

How this work was done

AI note. The authors chose the question, directed the work and judged every result. AI models from two developers assisted. The question, whether a single rational probabilistic finite automaton can have a noncomputable value and whether the values are exactly the left-c.e. reals, was posed to OpenAI's GPT-6.1 Sol, with the theorem of Hochman and Meyerovitch as the model to aim at. In a long working session, several agents of that model family proposed routes, attacked them and audited one another. GPT-6.1 Sol derived the construction and the proofs of Theorems A, B and C, Corollary D, Proposition 8.1(i),(ii) and Propositions 9.1 and 9.4, posed the question of three states, and recorded the routes that failed, among them that uniform undecidability does not by itself produce one noncomputable value, and that

summing rewards over witnessed computations cannot prevent a profitable certificate from being repeated. Before a result was kept, an agent that had not written it re-derived it refutation-first; the range theorem was re-derived independently by three agents, and the bound on the number of states by two. The literature lookups of the session were read-only.

Anthropic's Claude Opus 5.5 then checked the result refutation-first, as a review from the other developer. It re-derived each step from the notes of the session, before reading the session's own literature notes, and found no gap. It identified the compilation of an output machine into counter programs, and the universal program used for the bound on the number of states, as the steps written at the level of a sketch, and it noted that the rule for accepting and rejecting appeared in slightly different, equivalent forms across the notes. It judged the bookkeeping of the split test the most delicate step specific to the construction, and implemented the checking automaton and the payout for one small counter program as an explicit finite stochastic transition function in exact rational arithmetic; this test is described under Verification status. A separate run of Claude Opus 5.5 searched the literature for prior results, and another run wrote this manuscript. In writing, Section 3 and the index check of Section 7 were written out in full and one form of the payout was fixed. The following were added in writing: Remarks 6.4 and 6.5; the reading of Theorem C as undecidability of strict emptiness for A_*; the bound on the number of states in Proposition 8.1(iii) and the remark after Proposition 8.1; Proposition 9.5; and, in Section 9, the questions on fixed matrices and probabilistic Büchi automata, a question on channels that the later review answered (below), and the remark on quantum automata. A separate agent of Claude Opus 5.5 that had not written the text read it as a hostile referee, checking every statement against its proof and every attribution against the literature notes. It found one error in a stated lemma: the parser of Lemma 4.4, as first written, accepted certificates with repeated final rows. The error did not affect any bound, since such rows only lower the probability that the joint test survives, and it is corrected in this version, together with the attributions and wordings that the reading flagged.

A fresh session of GPT-6.1 Sol then reviewed the written manuscript refutation-first, without the notes of the original session. It re-derived Theorems A–D from the text and found no error in them. It observed that the question on channels in the first draft is answered by Theorem C together with [8, Theorem 1]; that observation is now Corollary 9.7, which Claude Opus 5.5 checked against the primary source. It also flagged the attribution to Boche, Schaefer and Poor, the scope of the remark on quantum automata, the abstract's description of the initial distribution and several wordings, all corrected here.

Models are not listed as authors; the authors take responsibility for every claim. The proofs are given in full, and none of them uses a computation.

Verification status

Theorems A, B and C, Corollary D and Proposition 8.1(i),(ii) were checked analytically, at the level of the notes of the working session, by the re-derivations within the developing model family and by the refutation-first review from the other developer described above; Propositions 9.1 and 9.4 were checked by the same review in passing. The written proofs rest on Sections 3 and 7, whose written form, like the statements added in writing listed above, was checked by the hostile reading of the text, by a model of the family that wrote it, and then by the review of the written manuscript from the developer of the model that found the construction, described above. That review also tested the certificate format as written here,

without control-state letters and with the corrected parser, in exact rational arithmetic: on one two-counter program and five choices of parameters it checked every reachable transition row, the masses of both tests, the gap on dishonest certificates, the payout and restart, the per-round decomposition on adversarial words and the repetition formula, with a planted dishonest certificate as a positive control, and in two exhaustive extensions it compared all 915,543 well-formed certificates in a fixed finite box (7 honest, 915,536 dishonest) against the full stochastic model, with no failures. Like the earlier test, it does not test the compilation of Section 3 or the universal program.

The finite-state implementation was tested in exact rational arithmetic during the review, before this text was written. The tested automaton has the payout (P3) and the index check (I1)–(I4) in the form written here; its certificates also list the control state of each row, which the format of Definition 4.2 omits. One test program had two counters, and its halting runs output exactly the binary words with an even number of ones; with the index check of Section 7 at $e = 1$, those with an odd number. The checking automaton, both tests with their slots, the register, the payout and the restart were built as an explicit transition function for five choices of (B, a, b) and of the index check, namely $(4, 1/8, 1/2)$ and $(16, 1/8, 1/2)$ without it, $(4, 1/8, 1/2)$ with $e = 0$ and with $e = 1$, and $(3, 1/5, 3/5)$ with $e = 2$; these parameters need not satisfy (1), which the test does not use. Every transition row that was used was nonnegative with sum exactly 1. For each choice, on all valid certificates with at most four output bits (15 or 16 per choice), the surviving probabilities of the two tests were equal and agreed with the products of Lemma 4.5, and the probabilities entering yes and no agreed with Lemma 5.1. On 83 to 95 certificates per choice that were well formed but not valid, including certificates with a wrong initial index, $\Psi \geq \kappa_B \Phi$ held. On 150 further words per choice, mixing up to four certificates with truncations and inserted or appended letters, the acceptance probability equalled the sum in the proof of Lemma 5.2 and was at most the largest ratio $\mathrm{acc}(t)/(\mathrm{acc}(t) + \mathrm{rej}(t))$ of its certificates, and the formula of Lemma 5.3 held for $m = 1, 2, 5$. There were no failures. About 1,400 states outside $\{\mathsf{yes}, \mathsf{no}\}$ were reachable for each choice. The test checks the finite-state bookkeeping on one program; it does not test the compilation of Section 3 or the universal program, and no proof uses it. Reports of the checks are available from the authors.

Credit

Probabilistic finite automata are due to Rabin [21]. The verification of counter-machine runs by unary coin-flipping tests is due to Condon and Lipton [6], building on an equality test of Freivalds [11]; we follow the accounts of Madani, Hanks and Condon [15, 16] and Rote [22]. The undecidability of emptiness is due to Nasu and Honda [19]; it is often credited to Paz's book [20], and [22] gives its history. The simulation of Turing machines by counter machines is Minsky's [17, 18]. Reals defined from a computably enumerable undecidable set, as α_* is, go back to Specker [25]. The model of a range theorem is that of Hochman and Meyerovitch [13].

References

[1] C. Baier, N. Bertrand and M. Größer, *On decision problems for probabilistic Büchi automata*, in *Foundations of Software Science and Computational Structures (FoSSaCS 2008)*, Lecture Notes in Computer Science **4962**, Springer, 2008, 287–301.

[2] V. D. Blondel, E. Jeandel, P. Koiran and N. Portier, *Decidable and undecidable problems about quantum automata*, SIAM Journal on Computing **34** (2005), 1464–1473.

[3] H. Boche, R. F. Schaefer and H. V. Poor, *Shannon meets Turing: non-computability and non-approximability of the finite state channel capacity*, Communications in Information and Systems **20** (2020), 81–116. arXiv:2008.13270.

[4] C. S. Calude, P. H. Hertling, B. Khoussainov and Y. Wang, *Recursively enumerable reals and Chaitin* Ω *numbers*, Theoretical Computer Science **255** (2001), 125–149.

[5] G. J. Chaitin, *A theory of program size formally identical to information theory*, Journal of the ACM **22** (1975), 329–340.

[6] A. Condon and R. J. Lipton, *On the complexity of space bounded interactive proofs (extended abstract)*, in *30th Annual Symposium on Foundations of Computer Science*, IEEE, 1989, 462–467.

[7] R. G. Downey and D. R. Hirschfeldt, *Algorithmic Randomness and Complexity*, Springer, New York, 2010.

[8] D. Elkouss and D. Pérez-García, *Memory effects can make the transmission capability of a communication channel uncomputable*, Nature Communications **9** (2018), 1149.

[9] N. Fijalkow, *Undecidability results for probabilistic automata*, ACM SIGLOG News **4**(4) (2017), 10–17.

[10] N. Fijalkow, H. Gimbert, E. Kelmendi and Y. Oualhadj, *Deciding the value 1 problem for probabilistic leaktight automata*, Logical Methods in Computer Science **11**(2) (2015), paper 12. arXiv:1504.04136.

[11] R. Freivalds, *Probabilistic two-way machines*, in *Mathematical Foundations of Computer Science 1981*, Lecture Notes in Computer Science **118**, Springer, 1981, 33–45.

[12] H. Gimbert and Y. Oualhadj, *Probabilistic automata on finite words: decidable and undecidable problems*, in *Automata, Languages and Programming (ICALP 2010)*, Lecture Notes in Computer Science **6199**, Springer, 2010, 527–538.

[13] M. Hochman and T. Meyerovitch, *A characterization of the entropies of multidimensional shifts of finite type*, Annals of Mathematics **171** (2010), 2011–2038.

[14] A. Kučera and T. A. Slaman, *Randomness and recursive enumerability*, SIAM Journal on Computing **31** (2001), 199–211.

[15] O. Madani, S. Hanks and A. Condon, *On the undecidability of probabilistic planning and infinite-horizon partially observable Markov decision problems*, in *Proceedings of the Sixteenth National Conference on Artificial Intelligence (AAAI-99)*, AAAI Press, 1999, 541–548.

[16] O. Madani, S. Hanks and A. Condon, *On the undecidability of probabilistic planning and related stochastic optimization problems*, Artificial Intelligence **147** (2003), 5–34.

[17] M. L. Minsky, *Recursive unsolvability of Post's problem of "Tag" and other topics in theory of Turing machines*, Annals of Mathematics **74** (1961), 437–455.

[18] M. L. Minsky, *Computation: Finite and Infinite Machines*, Prentice-Hall, Englewood Cliffs, 1967.

[19] M. Nasu and N. Honda, *Mappings induced by PGSM-mappings and some recursively unsolvable problems of finite probabilistic automata*, Information and Control **15** (1969), 250–273.

[20] A. Paz, *Introduction to Probabilistic Automata*, Academic Press, New York, 1971.

[21] M. O. Rabin, *Probabilistic automata*, Information and Control **6** (1963), 230–245.

[22] G. Rote, *Probabilistic finite automaton emptiness is undecidable*, 2024. arXiv:2405.03035.

[23] G. Rote, *Probabilistic finite automaton emptiness is undecidable for a fixed automaton*, in *50th International Symposium on Mathematical Foundations of Computer Science (MFCS 2025)*, Leibniz International Proceedings in Informatics **345**, 2025, paper 86. arXiv:2412.05198.

[24] R. I. Soare, *Turing Computability: Theory and Applications*, Springer, Berlin, 2016.

[25] E. Specker, *Nicht konstruktiv beweisbare Sätze der Analysis*, Journal of Symbolic Logic **14** (1949), 145–158.

[26] A. M. Turing, *On computable numbers, with an application to the Entscheidungsproblem*, Proceedings of the London Mathematical Society (2) **42** (1936–37), 230–265.